CMSC131: Loops, Random Numbers, and Patterns

Chapter 4

By Naren Ayinala

May 2026

Contents

1 Why Do We Need Loops? 3

2 The for Loop: Every Part Explained 3

2.1 Initialization: `int i = 1` . . . 3

2.2 Condition: `i <= 5` . . . 3

2.3 Update: `i++` . . . 4

2.4 A Complete Trace . . . 4

2.5 Changing the Starting Point, Step, and Direction . . . 4

2.6 A Sum and a Product . . . 5

3 The while Loop 5

3.1 Why while Is Useful When We Do Not Know the Count . . . 5

3.2 A while Loop May Run Zero Times . . . 5

4 Rewriting a for Loop as a while Loop 6

5 The do-while Loop 6

5.1 A Menu That Must Display at Least Once . . . 7

6 Comparing for, while, and do-while 7

7 break: Leave the Loop Immediately 8

7.1 break in Nested Loops . . . 8

8 continue: Skip the Rest of This Iteration 8

8.1 break Versus continue: Same Condition, Different Result . . . 8

9 Nested Loops: A Loop Inside Another Loop 9

9.1 Tracing the Inner Loop . . . 9
9.2 Increasing Triangle . . . 9
9.3 Decreasing Triangle . . . 10
9.4 Number Triangle . . . 10
9.5 Multiplication Table . . . 10
9.6 Right-Aligned Triangle . . . 11

10 Random Numbers: What Random Really Means 11

10.1 Dice and Coins . . . 11
10.2 Any Inclusive Integer Range . . . 12
10.3 Random and Loops Together . . . 12
10.4 Fixed Seeds and Reproducible Results . . . 12
10.5 Math.random() as Another Option . . . 12
10.6 A Guessing Game . . . 13

11 Common Loop Errors, Explained 13

11.1 Off-by-One Errors . . . 13
11.2 Accidental Semicolon . . . 13
11.3 An Infinite Loop . . . 14
11.4 One-Line Loop Bodies and Braces . . . 14

12 Final Worked Traces 14

12.1 Trace 1: Postincrement Inside the Body . . . 14
12.2 Trace 2: continue and the Update . . . 14
12.3 Trace 3: Nested Loop with break . . . 15

1 Why Do We Need Loops?

Imagine printing the numbers from 1 through 100. We could write one hundred separate `println` statements, but that would be tedious, difficult to change, and easy to get wrong. A **loop** lets us describe a repeated action once and specify when it should stop. We might want to print numbers, count occurrences, repeatedly ask for valid input, or draw a pattern. These are all variations of the same idea: *perform an action repeatedly while some rule permits it.*

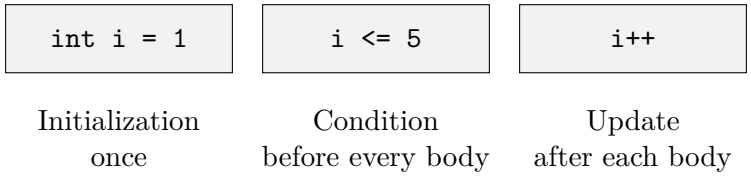
```
System.out.println(1);
System.out.println(2);
System.out.println(3);
// Instead, use a loop:
for (int number = 1; number <= 3; number++) {
    System.out.println(number);
}
```

A useful way to plan any loop is to identify three things: **starting state**, **continuation condition**, and **progress**. Without progress, the condition may never become false and the program may loop forever.

2 The for Loop: Every Part Explained

The usual counting loop has three expressions in its header:

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```



2.1 Initialization: `int i = 1`

This happens **once**, when execution first reaches the loop. It creates the counter and assigns its initial value. In this example, `i` starts at 1. Because it is declared in the loop header, it is not available after the loop ends.

2.2 Condition: `i <= 5`

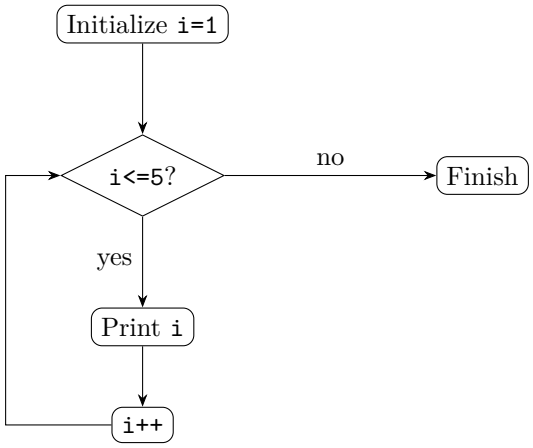
Java checks the condition **before** running the body. If it is true, Java runs the body. If false, Java skips the body and continues after the loop. The condition is checked again after each update. That final unsuccessful check is part of the execution even though it prints nothing.

2.3 Update: `i++`

After the body finishes normally, Java executes the update. Here `i++` adds one to the counter. It is not executed before the first iteration. For a simple standalone update, `++i` and `i++` have the same effect; their distinction matters when their values are used inside a larger expression.

2.4 A Complete Trace

Check	Value of `i`	`i <= 5`	What happens
1	1	true	Print 1, then increment to 2
2	2	true	Print 2, then increment to 3
3	3	true	Print 3, then increment to 4
4	4	true	Print 4, then increment to 5
5	5	true	Print 5, then increment to 6
6	6	false	Leave loop



2.5 Changing the Starting Point, Step, and Direction

```
for (int i = 0; i < 5; i++) { // 0 1 2 3 4
    System.out.print(i + " ");
}
for (int i = 2; i <= 10; i += 2) { // 2 4 6 8 10
    System.out.print(i + " ");
}
for (int i = 5; i >= 1; i--) { // 5 4 3 2 1
    System.out.print(i + " ");
}
```

For a descending loop, the counter decreases and the comparison generally reverses. If we accidentally write `i++` with `i >= 1` starting from 5, the condition will not become false through normal counting (integer overflow can eventually affect the result). This is why the update and condition must work together.

2.6 A Sum and a Product

A loop can accumulate results rather than merely print numbers.

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
    sum += i;
}
System.out.println(sum); // 10

int product = 1;
for (int i = 1; i <= 4; i++) {
    product *= i;
}
System.out.println(product); // 24
```

We start a sum at 0 because adding zero changes nothing. We start a product at 1 because multiplying by one changes nothing. Starting the product at 0 would keep it at 0 forever.

3 The while Loop

A `while` loop checks its condition before each iteration, just like a `for` loop. The difference is mainly how we organize the code: initialization and update usually appear outside the header.

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

Execution begins with `i = 1`. Java checks `i <= 5`. If true, it prints and increments. Then it goes back to check the condition. When `i` reaches 6, the condition is false and the loop stops.

3.1 Why while Is Useful When We Do Not Know the Count

A `for` loop is often convenient when we know how many iterations we expect. A `while` loop is especially readable when repetition depends on an event, such as finding a valid input or waiting until a target is reached.

```
int balance = 25;
while (balance < 100) {
    balance += 15;
    System.out.println(balance);
}
// 40 55 70 85 100
```

3.2 A while Loop May Run Zero Times

```
int x = 10;
while (x < 5) {
    System.out.println("Never printed");
```

```
}
System.out.println("Done"); // Done
```

Because the condition is false at the first check, the body never runs.

4 Rewriting a for Loop as a while Loop

The following two loops print exactly the same sequence. **Using for**

```
for (int i = 1; i <= 4; i++) {
    System.out.print(i + " ");
}
```

Using while

```
int i = 1;
while (i <= 4) {
    System.out.print(i + " ");
    i++;
}
```

`for` component	`while` location
Initialization	Before the `while`
Condition	In the `while (...)`
Update	Usually at the end of the body

There are two important subtleties. First, a variable declared in a `for` header has a narrower scope than a variable declared before a `while`. Second, if the loop body contains `continue`, a direct rewrite can behave differently: in a `for` loop, `continue` still performs the header's update; in a `while` loop, it jumps straight to the condition, potentially skipping an update written at the end.

```
// Safe rewrite of a for loop with continue:
int i = 0;
while (i < 5) {
    if (i == 2) {
        i++; // progress before continue
        continue;
    }
    System.out.print(i + " ");
    i++;
}
// 0 1 3 4
```

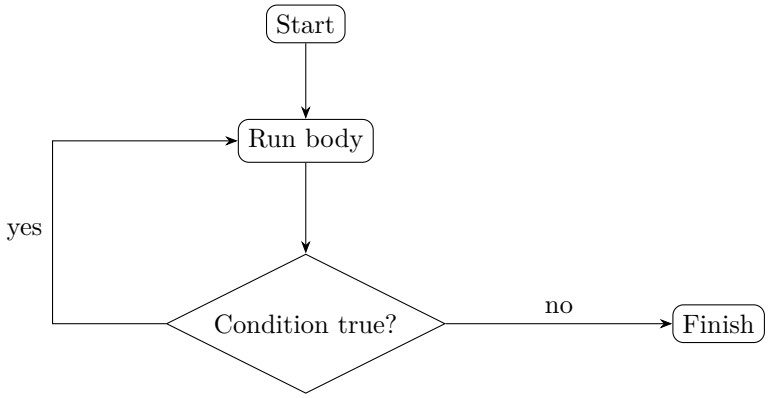
5 The do-while Loop

A `do-while` loop checks its condition **after** running the body. This guarantees the body runs at least once, even if the condition is false from the beginning.

```
int x = 10;
do {
```

```
    System.out.println(x); // prints 10 once
    x++;
} while (x < 5);
```

Notice the semicolon after `while (condition);`. That semicolon is part of the `do-while` syntax.



5.1 A Menu That Must Display at Least Once

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int choice;
do {
    System.out.println("1. Play");
    System.out.println("0. Exit");
    choice = sc.nextInt();
} while (choice != 0);
```

A menu should appear before the user has made any choice, so a `do-while` naturally expresses the requirement. A `while` loop can implement the same behavior, but would need an appropriate initial value or a first read before the loop.

6 Comparing for, while, and do-while

Question	`for`	`while`	`do-while`
When tested?	Before body	Before body	After body
Minimum executions	0	0	1
Typical use	Counting	Condition-driven repetition	Menu/input-at-least-once
Where is update?	Usually header	Usually body	Usually body

These are conventions, not strict limitations. All three forms can express many of the same algorithms. Choose the one that communicates the intended repetition most clearly.

7 break: Leave the Loop Immediately

`break` ends the nearest enclosing loop immediately. Execution resumes at the first statement after that loop. The loop's normal condition need not have become false.

```
for (int i = 1; i <= 10; i++) {
    if (i == 4) {
        break;
    }
    System.out.print(i + " ");
}
// 1 2 3
```

When `i` becomes 4, the `if` condition is true and `break` exits the loop before the print. The loop update is not executed after `break`.

7.1 break in Nested Loops

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 4; col++) {
        if (col == 3) break;
        System.out.print(col);
    }
    System.out.println();
}
// 12
// 12
// 12
```

The `break` exits the *inner* loop only. The outer loop still advances to its next row.

8 continue: Skip the Rest of This Iteration

`continue` does not end the whole loop. It skips the remaining statements in the current iteration and proceeds to the next iteration. In a `for` loop, the update happens before the next condition check.

```
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) {
        continue;
    }
    System.out.print(i + " ");
}
// 1 3 5
```

8.1 break Versus continue: Same Condition, Different Result

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.print(i + " ");
}
// 1 2
```

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.print(i + " ");
}
// 1 2 4 5
```

A common error is to place the counter update after a `continue` in a `while` loop. If the update is skipped every time, the loop may become infinite.

```
int i = 0;
while (i < 4) {
    i++; // update before any possible continue
    if (i == 2) continue;
    System.out.print(i + " ");
}
// 1 3 4
```

9 Nested Loops: A Loop Inside Another Loop

A nested loop runs its inner loop for each iteration of the outer loop. The inner loop normally starts over from its initialization every time the outer loop advances.

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 4; col++) {
        System.out.print("*");
    }
    System.out.println();
}
// ****
// ****
// ****
```

The outer loop controls the number of rows. The inner loop controls the number of characters printed in each row. `print()` stays on the same line; `println()` moves to the next line.

9.1 Tracing the Inner Loop

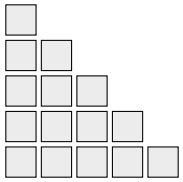
Outer row	Inner values	Printed on that row
1	1,2,3,4	****
2	1,2,3,4	****
3	1,2,3,4	****

9.2 Increasing Triangle

```
for (int row = 1; row <= 5; row++) {
    for (int col = 1; col <= row; col++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
// *
// **
// ***
// ****
// *****
```

Unlike the rectangle, the inner loop's limit depends on `row`. The first row prints one star; the fifth prints five.



9.3 Decreasing Triangle

```
for (int row = 5; row >= 1; row--) {
    for (int col = 1; col <= row; col++) {
        System.out.print("*");
    }
    System.out.println();
}
// *****
// ****
// ***
// **
// *
```

9.4 Number Triangle

```
for (int row = 1; row <= 4; row++) {
    for (int col = 1; col <= row; col++) {
        System.out.print(col);
    }
    System.out.println();
}
// 1
// 12
// 123
// 1234
```

9.5 Multiplication Table

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 3; col++) {
        System.out.print((row * col) + " ");
    }
    System.out.println();
}
// 1 2 3
```

```
// 2 4 6
// 3 6 9
```

9.6 Right-Aligned Triangle

Print leading spaces before stars. For row `r` of a height-5 triangle, we need `5-r` spaces and `r` stars.

```
for (int row = 1; row <= 5; row++) {
    for (int space = 1; space <= 5 - row; space++) {
        System.out.print(" ");
    }
    for (int star = 1; star <= row; star++) {
        System.out.print("*");
    }
    System.out.println();
}
// *
// **
// ***
// ****
// *****
```

10 Random Numbers: What Random Really Means

Many programs need outcomes that are not chosen directly by the programmer: rolling dice, selecting a random quiz question, or simulating a game. Java provides a `Random` class for **pseudorandom** values. Pseudorandom means a deterministic algorithm produces values that behave like random values for ordinary simulation; a fixed seed reproduces the same sequence.

```
import java.util.Random;
Random random = new Random();
int value = random.nextInt(10);
// Any integer from 0 through 9
```

The argument to `nextInt(bound)` is an *exclusive* upper bound. For positive `bound`, possible values are `0` through `bound - 1`. This is one of the most important details to remember.

10.1 Dice and Coins

```
Random random = new Random();
int die = random.nextInt(6) + 1; // 1 through 6
int coin = random.nextInt(2); // 0 or 1
if (coin == 0) {
    System.out.println("Heads");
} else {
    System.out.println("Tails");
}
```

Adding 1 shifts the range from 0–5 to 1–6 without changing how many possible values exist.

10.2 Any Inclusive Integer Range

To generate integers from `low` through `high`, including both endpoints, the standard beginner formula is

`low + random.nextInt(high - low + 1)`.

The bound `high-low+1` counts how many integers lie in the interval. The added `low` shifts the first possible result to the desired minimum. Assume the range is valid and its width fits in an `int`.

Desired range	Java expression	Possible values
0–9	`nextInt(10)`	0,...,9
1–10	`1 + nextInt(10)`	1,...,10
5–12	`5 + nextInt(8)`	5,...,12
-3–3	`-3 + nextInt(7)`	-3,...,3
20–25	`20 + nextInt(6)`	20,...,25

In the table, `nextInt` refers to a call on a `Random` object, such as `random.nextInt(10)`.

10.3 Random and Loops Together

```
Random random = new Random();
for (int roll = 1; roll <= 5; roll++) {
    int die = random.nextInt(6) + 1;
    System.out.println("Roll " + roll + ": " + die);
}
```

The output changes from run to run, but every printed die value is between 1 and 6.

10.4 Fixed Seeds and Reproducible Results

```
Random first = new Random(42);
Random second = new Random(42);
for (int i = 0; i < 5; i++) {
    System.out.println(first.nextInt(100) ==
                       second.nextInt(100));
}
// true printed five times
```

Both generators use the same algorithm and seed, so their corresponding calls produce the same sequence. This is useful when debugging a simulation because a failure can be reproduced. A fixed seed does not make the results secure or unpredictable.

10.5 Math.random() as Another Option

`Math.random()` returns a `double` from 0.0 inclusive up to 1.0 exclusive. We can scale and cast it to get an integer range.

```
int digit = (int) (Math.random() * 10); // 0..9
int die = (int) (Math.random() * 6) + 1; // 1..6
```

The cast truncates the nonnegative value toward zero. Parentheses matter: `(int)Math.random() * 6` is always 0 because the cast happens before multiplication.

10.6 A Guessing Game

```
import java.util.Random;
import java.util.Scanner;

public class GuessingGame {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Random random = new Random();
        int secret = random.nextInt(10) + 1;
        int guess = 0;
        while (guess != secret) {
            System.out.print("Guess 1-10: ");
            guess = scanner.nextInt();
            if (guess < secret) {
                System.out.println("Too low");
            } else if (guess > secret) {
                System.out.println("Too high");
            } else {
                System.out.println("Correct!");
            }
        }
    }
}
```

This program demonstrates a `while` loop where we cannot predict the number of iterations: it depends on how many guesses the user needs.

11 Common Loop Errors, Explained

11.1 Off-by-One Errors

```
for (int i = 0; i < 5; i++) { // five iterations: 0..4
    System.out.print(i);
}
for (int i = 0; i <= 5; i++) { // six iterations: 0..5
    System.out.print(i);
}
```

Changing `<` to `<=` includes one more value. Always list the actual counter values before deciding how many iterations occur.

11.2 Accidental Semicolon

```
for (int i = 0; i < 3; i++); // empty loop body
{
    System.out.println("Hello"); // prints once
}
```

The semicolon immediately after the `for` header is a complete empty statement. The following braces form a separate block, not the loop body.

11.3 An Infinite Loop

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    // Missing i++: i stays 0 forever.
}
```

An infinite loop is sometimes intentional (for example, a server loop), but accidental infinite loops usually arise because the state never changes in a way that makes the condition false.

11.4 One-Line Loop Bodies and Braces

Like `if`, a loop without braces controls only the next statement.

```
for (int i = 0; i < 3; i++)
    System.out.print(i);
System.out.println("Done"); // after the loop
```

Braces are strongly recommended when teaching and when adding more statements later, so the loop's scope is unambiguous.

12 Final Worked Traces

12.1 Trace 1: Postincrement Inside the Body

```
int x = 0;
while (x < 3) {
    System.out.print(x++ + " ");
}
System.out.println("end=" + x);
// 0 1 2 end=3
```

The expression `x++` supplies the old value for printing and then increments `x`. Thus the first print uses 0, even though `x` becomes 1 immediately afterward.

12.2 Trace 2: continue and the Update

```
int sum = 0;
for (int i = 1; i <= 6; i++) {
    if (i % 3 == 0) continue;
    sum += i;
}
System.out.println(sum); // 12 (1+2+4+5)
```

12.3 Trace 3: Nested Loop with break

```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 4; j++) {
        if (j == 3) break;
        System.out.print(i + "-" + j + " ");
    }
}
// 1-1 1-2 2-1 2-2
```

The outer loop still reaches `i = 2`; each inner loop exits when `j` reaches 3.